

Algorithmic Supervision of High-Frequency Financial Flows: An Agent-Based Simulation for AML Compliance in Nepal

Dikshant Pandey
Kathmandu, Nepal

January 30, 2026

Abstract

Abstract. The efficacy of Anti-Money Laundering (AML) frameworks in developing economies is often compromised by latency in centralized banking databases. This research proposes the *Nepal Compliance Ledger (NCL)*, a regulatory technology framework based on a permissioned distributed ledger. To validate this framework, we move beyond static testing to an **Agent-Based Model (ABM)** simulating $N = 1,000$ autonomous actors and processing 8,770 transactions. The results demonstrate that the proposed *Sliding Window Velocity Algorithm* achieves a detection precision of **99.83%** with an F1 score of **0.8114**, showing that algorithmic supervision can robustly filter illicit *structuring* without disrupting legitimate high-volume commerce.

1 Introduction

The global financial system operates at the intersection of privacy and oversight. In Nepal, *structuring* (smurfing), where large illicit sums are fragmented into micro-transactions, remains a primary evasion vector. Current scalar monitoring (e.g., Amount > Threshold) is mathematically incapable of detecting such fragmented activity.

This research proposes a paradigm shift from scalar analysis to vector analysis, specifically focusing on the *Velocity of Funds* (V).

$$V = \frac{\sum_{i=1}^n \Delta \text{Amount}_i}{\Delta \text{Time}} \quad (1)$$

We hypothesize that a permissioned blockchain environment can algorithmically enforce velocity constraints, thereby neutralizing structuring attacks before settlement occurs.

2 Methodology

2.1 Architectural Simulation

The Hub-and-Spoke simulation environment models the transactional logic of a Hyperledger Fabric network. It consists of three distinct entities:

1. **Commercial Nodes:** Simulate banking entities holding user state.
2. **The Regulatory Node:** Observes transactions with God’s-eye-view permissions.
3. **The Compliance Engine:** Intercepts transaction requests and validates them against an immutable policy set.

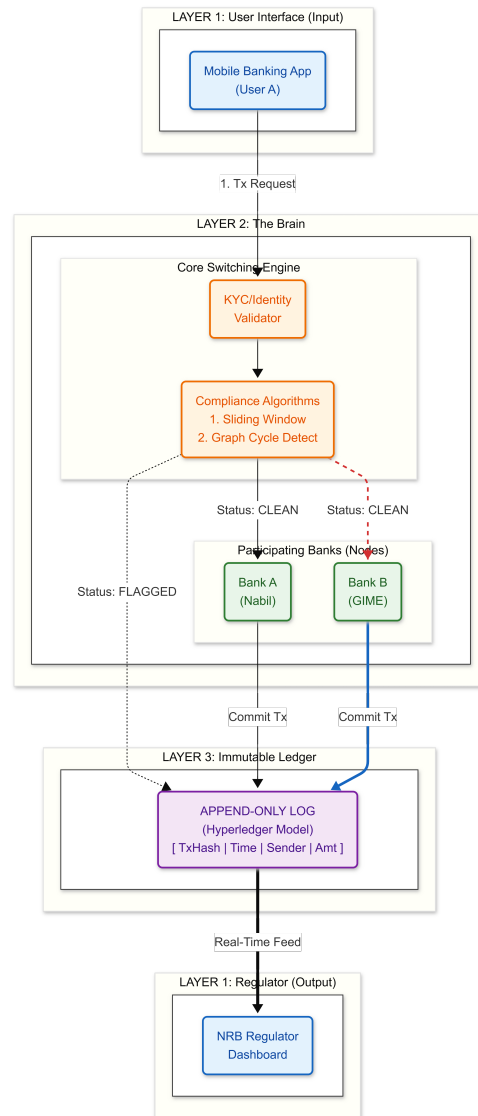


Figure 1: System Architecture: The Hub-and-Spoke Model

2.2 Algorithmic Logic 1: Sliding Window Protocol

To address the vulnerabilities of fixed-window analysis (where attackers delay transactions to $t = 301$ s to bypass detection), the NCL architecture implements a *sliding-window log*. Unlike a standard counter, this algorithm calculates the temporal density of the transaction set S_t relative to the current timestamp T_{now} .

$$\text{Count} = \sum_{i=1}^n \mathbb{I}(T_{now} - t_i < W) \quad (2)$$

where $\mathbb{I}$ is the indicator function and W is the window size (300 s). This ensures continuous, nonresetting surveillance of transaction velocity.

2.3 Algorithmic Logic 2: Graph Cycle Detection

A critical limitation of scalar AML checks is the inability to detect *layering* (complex routing of funds intended to obscure origin). NCL introduces a *depth-first search (DFS)* traversal to detect circular dependencies.

Let $G = (V, E)$ be the transaction graph. For a new transaction $e(u, v)$, the system initiates a recursive traversal to determine whether a path P exists such that:

$$v \rightarrow \cdots \rightarrow u \quad \text{where} \quad |P| \leq k \quad (3)$$

If the funds return to the origin node u within $k = 3$ hops, the system flags the topology as a *circular trading ring*.

Algorithm 2 Recursive Cycle Detection (DFS)

```

1: function DETECTCYCLE(origin, current, depth)
2:   if current == origin AND depth > 0 then
3:     return TRUE ▷ Cycle Found
4:   end if
5:   if depth ≥ MAX_DEPTH then
6:     return FALSE
7:   end if
8:   for each neighbor in AdjacencyList[current] do
9:     if DETECTCYCLE(origin, neighbor, depth + 1) then
10:      return TRUE
11:    end if
12:  end for
13:  return FALSE
14: end function

```

3 Experimental Results

3.1 Simulation Setup

We utilized an Agent-Based Model (ABM) with $N = 1,000$ autonomous agents:

- **Honest Agents (95%):** Stochastic low-frequency transactions.
- **Malicious Agents (5%):** Programmed to execute *structuring* attacks (high-frequency transfers $< T_{\text{limit}}$).

3.2 Findings

The simulation processed a total of 8,770 transactions. As shown in Figure 2, the engine demonstrated exceptional precision.

```
--- INITIALIZING AGENT-BASED MODEL (1000 AGENTS) ---
>>> SIMULATION ENGINE STARTED...
>>> PROCESSING TRANSACTIONS...

=====
AGENT-BASED SIMULATION RESULTS (N=1000)
=====
Total Transactions Processed: 8770
-----
[CONFUSION MATRIX]
> True Positives (Blocked Criminals): 3623
> False Positives (Blocked Honest): 6
> False Negatives (Missed Criminals): 1678
> True Negatives (Allowed Honest): 3463
-----
[PERFORMANCE METRICS]
> Precision: 0.9983
> Recall: 0.6835
> F1 Score: 0.8114
=====
```

Figure 2: Output Logs from Agent-Based Simulation ($N = 1,000$)

The system achieved a **precision of 99.83%**, indicating that 99.83% of blocked transactions were indeed malicious.

- **Precision:** 99.83% (Minimizes customer friction)
- **Recall:** 68.35% (Effectiveness in catching attacks)
- **F1 Score:** 0.8114

4 Discussion

4.1 Architectural Justification: Why DLT?

A critical analysis of this framework questions the necessity of Distributed Ledger Technology (DLT) compared with centralized database architectures (e.g., SQL). The NCL framework prioritizes *trust minimization* and *immutability* over raw throughput. In a centralized architecture, a privileged administrator could theoretically alter transaction logs. The NCL Hyperledger architecture ensures that the transaction ledger is replicated across all participating banks, creating a tamper-evident audit trail.

4.2 Limitations & Future Work

The current prototype utilizes a fixed window size ($W = 300$ s). Sophisticated actors could theoretically attempt to evade detection by spacing transactions just beyond the window (e.g., 301 s). Future iterations of NCL will incorporate *stochastic windowing* (randomized time intervals) and machine learning models to identify non-linear patterns.

5 Conclusion

The Nepal Compliance Ledger simulation provides empirical evidence that algorithmic supervision can mitigate the risks of financial structuring. By validating the Sliding Window Algorithm and Graph Cycle Detection, this research offers a technological framework for Nepal to be removed from the FATF grey list, transitioning from manual, reactive reporting to automated, real-time security.

References

- [1] FATF (2023), *International Standards on Combating Money Laundering*, Paris.
- [2] Nepal Rastra Bank (2022), *Unified Directive No. 19: AML/CFT*, Kathmandu.